

jev-decision-service 0.1.4 · 验收报告

验收依据：上一轮提交的《六项缺陷定位与修复》 | 真机环境：NVIDIA DGX Spark (GB10) + ollama + Qwen3-8B | 2026-09-27

0 结论

0.1.4 把我上一轮报的 6 项缺陷全部修复了，其中 3 处采用了比我的建议更好的做法。真机实测 opensource 路径从 0/3 变成 3/3，单元测试从 29 增至 35，全绿。

6 / 6

缺陷修复

3 / 3

真机成功率

35

单元测试全过

+219

代码行数增量

结论：我之前提供的 patch_jev_service.py 已不再需要——0.1.4 是原生修复，不是靠打补丁。补丁脚本已标注「仅适用于 0.1.0~0.1.3」。

1 逐条验收

#	上轮报的问题	0.1.4 的修复	评价
1	提示词未要求原生键名，与解析器不匹配	system prompt 明确写 “Use the exact native keys: choice for Choice, score for Score, and noul for Noul. Do not use a generic value key”，并补充「概率可近似、会被归一化」「score 从 0 开始」	修复，说明更完整
2	概率和不严格为 1 就报错	_normalize_raw_answers 统一归一化，并记录 {id}:probabilities_normalized 审计警告	修复，且新增审计轨迹
3	score 越界报错	支持 score_label_to_index (选项名转下标) + 越界钳制	修复
4	payload 未设 max_tokens，多问 JSON 被截断	max_tokens: int = 1024，下发时做 min(max(1, ...), 4096) 限幅	修复，且更安全
5	启用校准后 score 的 value 被改写成标签名，紧急度保护失效	按类型分支：choice 取 argmax 标签名，score 取概率加权期望值	★ 比建议更好
6	稀有类告警是静态全局的，声明即永久转人工	告警从全局下沉到 prediction.warnings；新增 class_support_head 指定适用预测；policy 仅在本次预测命中稀有类时拦截	★ 比建议更好
7	格式类错误 retryable=False，导致不降级	新增 _format_error()，统一返回 retryable=True	修复 (上轮附带建议)

2 三处比建议更好的实现

项	我的建议	0.1.4 的做法	为什么更好
score 校准后的取值	取概率最高档位的**下标** (arg max，如 3.0)	取**概率加权期望值** (如 2.808354)	保留了分布的不确定性信息。argmax 会丢掉「其实只有 51% 把握是最高档」这个信息，期望值则把犹豫体现在数值上
稀有类告警的归属	在 policy 里把预测标签与稀有类名单比对	告警下沉到每条 prediction.warnings，并用 class_support_head 指定哪个预测适用	职责更清晰：告警跟着数据走，而不是靠 policy 反查；也让「哪个决策受稀有类影响」一目了然
max_tokens 下发	直接传 self.max_tokens	传 min(max(1, self.max_tokens), 4096)	防止误配置 (如 0 或过大) 打穿后端

3 真机与测试实测数据

版本	opensource 成功率	单条耗时	单元测试
0.1.0	0 / 3	—	20
0.1.1	未单独测	—	26
0.1.3	0 / 3	—	29
0.1.3 + 我的补丁	3 / 3	13.2 s	29
0.1.4 (原生)	3 / 3	17.0 s	35

0.1.4 单次耗时 17.0 s，比打补丁的 13.2 s 略长。原因应该是提示词更详细（明确列出键名、概率处理、score 起点），模型输出更规范但也更长。以「稳定可用」为目标的取舍是合理的。

3.1 两条严重缺陷的对照复测

缺陷	场景	0.1.3	0.1.4
5 · 紧急度保护	severity=3，calibrated=True	value='立即' (str)，保护失效	value=2.808 (float)，保护生效
6 · 稀有类拦截	健康设备 + 已声明稀有类	human_review + blocked	answer_from_context，不拦截

4 代码规模

文件	0.1.3	0.1.4	增量
provider.py	447	550	+103
policy.py	111	124	+13
calibration.py	89	97	+8
engine.py	144	147	+3
models.py	196	198	+2
questions.py	72	73	+1
README.md	94	98	+4
test_provider.py	82	148	+66
test_policy.py	76	99	+23
合计 (Python)	1895	2114	+219 / +11.6%

改动集中在 provider.py (+103) 与两个测试文件 (+89)，与缺陷分布完全对应——说明修复是定点的，没有波及无关模块。state_builder.py / memory.py / tools.py / server.py 等零改动。

5 一个值得讨论的设计点

缺陷 5 的修复把 score 的值从「档位」变成了「期望档位」（例如 2.808 而不是 3）。这在保留不确定性上更好，但会改变 policy 阈值的语义：

取值方式	含义	对阈值的影响
argmax 下标	「最可能的档位是第 3 档」	urgent_score=2.0 表示「最高档达到尽快/立即」

取值方式	含义	对阈值的影响
概率加权期望	「平均而言落在 2.8 档」	同样的 2.0 阈值现在意味着「分布整体偏重高档」，判据比原来更严

建议：如果 policy 的 urgent_score 是按「档位」调的，换用期望值后应重新核对阈值。例如分布 {观察 0.4, 计划 0.3, 尽快 0.2, 立即 0.1} 的期望值是 1.0、argmax 是「观察」(0)，两者触发结果不同。这不是缺陷，只是语义变化，需要在领域侧确认阈值仍然合适。

6 后续

项	状态
6 项缺陷	全部修复，真机验证通过
我的补丁 patch_jev_service.py	0.1.4 起不再需要 (已标注适用版本 0.1.0~0.1.3)
哨兵决策头桥接 sentinel_jev_bridge.py	在 0.1.4 上仍可正常运行 (校准状态 temperature_scaled)
建议下一步	把哨兵决策头作为 StructuredHeadProvider 正式接入，形成「框架 + 决策头」的完整系统；并补 StepFun 云端接入

对比基线：0.1.3 (1895 行 / 29 测试) → 0.1.4 (2114 行 / 35 测试) | 全部结论来自 gx10-902c 节点实测